

Guide de personnalisation et d'extension

Ce guide vous explique comment personnaliser et étendre votre application de gestion de truffière.

Personnalisation visuelle

Changer les couleurs principales

Éditez `frontend/src/App.css` :

```
css

/* Vert de la truffière - Remplacez par vos couleurs */
.navbar {
  background: linear-gradient(135deg, #2c5f2d 0%, #97bc62 100%);
}

.btn-primary {
  background: #2c5f2d;
}

.stat-card {
  background: linear-gradient(135deg, #2c5f2d 0%, #97bc62 100%);
}
```

Modifier le logo et le titre

Dans `frontend/src/App.js` :

```
javascript

<h1>  Votre Nom de Truffière</h1>
```

Changer les icônes

Remplacez les émojis par des icônes FontAwesome ou Material Icons :

```
bash

npm install @fortawesome/fontawesome-free
# ou
npm install @mui/icons-material
```

Ajouter de nouveaux champs

Exemple : Ajouter un champ "Responsable" aux parcelles

1. Modifier la base de données (`init-db.sql`) :

```
sql
```

```
ALTER TABLE parcelles ADD COLUMN responsable VARCHAR(100);
```

2. Appliquer la migration :

```
bash
```

```
docker exec -it truffiere_db psql -U truffiere_user -d truffiere << EOF
ALTER TABLE parcelles ADD COLUMN responsable VARCHAR(100);
EOF
```

3. Modifier l'API ((backend/server.js)) :

```
javascript
```

```
app.post('/api/parcelles', async (req, res) => {
  const { nom, surface_ha, type_sol, ph_sol, exposition, responsable, notes } = req.body;
  const result = await pool.query(
    'INSERT INTO parcelles (nom, surface_ha, type_sol, ph_sol, exposition, responsable, notes) VALUES ($1, $2, $3, $4, $5, $6, $7)'
  );
  res.status(201).json(result.rows[0]);
});
```

4. Modifier le formulaire ((frontend/src/components/Parcelles.js)) :

javascript

```
const [formData, setFormData] = useState({
  nom: "",
  surface_ha: "",
  type_sol: "",
  ph_sol: "",
  exposition: "",
  responsable: "", // Nouveau champ
  notes: ""
});
```

// Dans le formulaire, ajouter :

```
<div className="form-group">
  <label>Responsable</label>
  <input
    type="text"
    name="responsable"
    value={formData.responsable}
    onChange={handleInputChange}
    placeholder="Nom du responsable"
  />
</div>
```

Ajouter la cartographie

Intégrer Leaflet pour afficher les parcelles

1. Installer les dépendances (déjà fait) :

bash

```
docker-compose exec frontend npm install leaflet react-leaflet
```

2. Créer un composant Carte (`frontend/src/components/Carte.js`) :

javascript

```
import React from 'react';
import { MapContainer, TileLayer, Marker, Popup, Polygon } from 'react-leaflet';
import 'leaflet/dist/leaflet.css';
import L from 'leaflet';

// Fix icône par défaut de Leaflet
delete L.Icon.Default.prototype._getIconUrl;
L.Icon.Default.mergeOptions({
  iconRetinaUrl: 'https://cdnjs.cloudflare.com/ajax/libs/leaflet/1.7.1/images/marker-icon-2x.png',
  iconUrl: 'https://cdnjs.cloudflare.com/ajax/libs/leaflet/1.7.1/images/marker-icon.png',
  shadowUrl: 'https://cdnjs.cloudflare.com/ajax/libs/leaflet/1.7.1/images/marker-shadow.png',
});

function Carte({ parcelles, arbres }) {
  // Centre de la carte (coordonnées de votre truffière)
  const center = [47.0, 1.0]; // Remplacer par vos coordonnées

  return (
    <MapContainer
      center={center}
      zoom={15}
      style={{ height: '600px', width: '100%' }}
    >
      <TileLayer
        url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
        attribution='&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>'
      />

      { /* Afficher les arbres */ }
      {arbres.map(arbre => (
        arbre.position && (
          <Marker
            key={arbre.id}
            position={[arbre.position.lat, arbre.position.lng]}
          >
            <Popup>
              <strong>{arbre.numero}</strong><br/>
              {arbre.espece}<br/>
              État: {arbre.etat}
            </Popup>
          </Marker>
        )
      ))}
    </MapContainer>
  );
}
```

```
export default Carte;
```

3. Utiliser le composant :

```
javascript
```

```
import Carte from './components/Carte';
```

```
// Dans votre composant
```

```
<Carte parcelles={parcelles} arbres={arbres} />
```



Ajouter des graphiques avec Recharts

Exemple : Graphique de production mensuelle

1. Créer un composant Graphique :

```
javascript
```

```
import React from 'react';
```

```
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, Legend } from 'recharts';
```

```
function GraphiqueProduction({ data }) {
```

```
  // data = [{mois: 'Jan', production: 45}, {mois: 'Fév', production: 52}, ...]
```

```
  return (
```

```
    <div style={{ width: '100%', height: 400 }}>
```

```
      <h3>Production mensuelle (kg)</h3>
```

```
      <LineChart width={800} height={300} data={data}>
```

```
        <CartesianGrid strokeDasharray="3 3" />
```

```
        <XAxis dataKey="mois" />
```

```
        <YAxis />
```

```
        <Tooltip />
```

```
        <Legend />
```

```
        <Line
```

```
          type="monotone"
```

```
          dataKey="production"
```

```
          stroke="#2c5f2d"
```

```
          strokeWidth={2}
```

```
        />
```

```
      </LineChart>
```

```
    </div>
```

```
  );
```

```
}
```

```
export default GraphiqueProduction;
```



Ajouter l'upload de photos

Pour les arbres ou les récoltes

1. Modifier la base de données :

```
sql

ALTER TABLE arbres ADD COLUMN photo_url TEXT;
ALTER TABLE recoltes ADD COLUMN photos TEXT[]; -- Array de URLs
```

2. Ajouter un service de stockage :

Option A : Utiliser un volume Docker local

Option B : Intégrer un service cloud (AWS S3, Cloudinary, etc.)

3. Exemple avec upload local :

Dans `backend/server.js` :

```
javascript

const multer = require('multer');
const path = require('path');

// Configuration multer
const storage = multer.diskStorage({
  destination: './uploads/',
  filename: (req, file, cb) => {
    cb(null, Date.now() + path.extname(file.originalname));
  }
});

const upload = multer({ storage });

// Route d'upload
app.post('/api/upload', upload.single('photo'), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ error: 'Aucun fichier' });
  }
  res.json({
    url: `/uploads/${req.file.filename}`,
    filename: req.file.filename
  });
});

// Servir les fichiers uploadés
app.use('/uploads', express.static('uploads'));
```

4. Composant React pour l'upload :

javascript

```
function UploadPhoto({ onUpload }) {  
  const handleFileChange = async (e) => {  
    const file = e.target.files[0];  
    if (!file) return;  
  
    const formData = new FormData();  
    formData.append('photo', file);  
  
    try {  
      const response = await axios.post(`${API_URL}/upload`, formData, {  
        headers: { 'Content-Type': 'multipart/form-data' }  
      });  
      onUpload(response.data.url);  
    } catch (error) {  
      console.error('Erreur upload:', error);  
    }  
  };  
  
  return (  
    <div>  
      <input type="file" accept="image/*" onChange={handleFileChange} />  
    </div>  
  );  
}
```



Ajouter l'authentification

Avec JWT (JSON Web Tokens)

1. Installer les dépendances :

```
bash
```

```
docker-compose exec backend npm install jsonwebtoken bcrypt
```

2. Créer une table utilisateurs :

sql

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  username VARCHAR(50) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  email VARCHAR(100),  
  role VARCHAR(20) DEFAULT 'user',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

3. Routes d'authentification (`backend/server.js`) :

javascript

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');

const JWT_SECRET = process.env.JWT_SECRET || 'votre_secret_tres_securise';

// Inscription
app.post('/api/auth/register', async (req, res) => {
  const { username, password, email } = req.body;
  try {
    const hashedPassword = await bcrypt.hash(password, 10);
    const result = await pool.query(
      'INSERT INTO users (username, password_hash, email) VALUES ($1, $2, $3) RETURNING id, username, email',
      [username, hashedPassword, email]
    );
    res.status(201).json(result.rows[0]);
  } catch (error) {
    res.status(500).json({ error: 'Erreur lors de l\'inscription' });
  }
});
```

// Connexion

```
app.post('/api/auth/login', async (req, res) => {
  const { username, password } = req.body;
  try {
    const result = await pool.query(
      'SELECT * FROM users WHERE username = $1',
      [username]
    );

    if (result.rows.length === 0) {
      return res.status(401).json({ error: 'Identifiants invalides' });
    }

    const user = result.rows[0];
    const validPassword = await bcrypt.compare(password, user.password_hash);

    if (!validPassword) {
      return res.status(401).json({ error: 'Identifiants invalides' });
    }

    const token = jwt.sign(
      { id: user.id, username: user.username, role: user.role },
      JWT_SECRET,
      { expiresIn: '24h' }
    );

    res.json({ token, user: { id: user.id, username: user.username, role: user.role } });
  } catch (error) {
    res.status(500).json({ error: 'Erreur lors de la connexion' });
  }
});
```

```

    } catch (error) {
      res.status(500).json({ error: 'Erreur serveur' });
    }
  });

  // Middleware de vérification du token
  function authenticateToken(req, res, next) {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];

    if (!token) {
      return res.status(401).json({ error: 'Token manquant' });
    }

    jwt.verify(token, JWT_SECRET, (err, user) => {
      if (err) {
        return res.status(403).json({ error: 'Token invalide' });
      }
      req.user = user;
      next();
    });
  }

  // Protéger les routes
  app.get('/api/parcelles', authenticateToken, async (req, res) => {
    // ... votre code existant
  });

```

4. Frontend - Gestion du token :

javascript

// Créer un contexte d'authentification

```
import React, { createContext, useState, useContext } from 'react';
import axios from 'axios';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem('token'));

  const login = async (username, password) => {
    const response = await axios.post(`${API_URL}/auth/login`, { username, password });
    setToken(response.data.token);
    setUser(response.data.user);
    localStorage.setItem('token', response.data.token);

    // Configurer axios pour inclure le token
    axios.defaults.headers.common['Authorization'] = `Bearer ${response.data.token}`;
  };

  const logout = () => {
    setToken(null);
    setUser(null);
    localStorage.removeItem('token');
    delete axios.defaults.headers.common['Authorization'];
  };

  return (
    <AuthContext.Provider value={{ user, token, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```



Ajouter l'export PDF

Avec jsPDF

1. Installer la dépendance :

```
bash
```

```
docker-compose exec frontend npm install jsPDF jsPDF-autotable
```

2. Créer une fonction d'export :

javascript

```
import jsPDF from 'jspdf';
import 'jspdf-autotable';

function exportToPDF(data, filename) {
  const doc = new jsPDF();

  // En-tête
  doc.setFontSize(18);
  doc.text('Rapport de Production - Truffière', 14, 20);

  doc.setFontSize(11);
  doc.text(`Date: ${new Date().toLocaleDateString('fr-FR')}`, 14, 30);

  // Tableau des données
  doc.autoTable({
    startY: 40,
    head: [['Parcelle', 'Année', 'Production (kg)', 'Valeur (€)']],
    body: data.map(row => [
      row.parcelle,
      row.annee,
      (row.poids_total_g / 1000).toFixed(2),
      parseFloat(row.valeur_totale || 0).toFixed(2)
    ]),
    theme: 'grid',
    headStyles: { fillColor: [44, 95, 45] }
  });

  // Sauvegarder
  doc.save(filename || 'rapport.pdf');
}

// Utilisation
<button onClick={() => exportToPDF(stats.parcelles, 'production.pdf')}>
  📄 Exporter en PDF
</button>
```

🔔 Ajouter des notifications

Avec react-toastify

1. Installer :

```
bash
```

```
docker-compose exec frontend npm install react-toastify
```

2. Configurer dans App.js :

javascript

```
import { ToastContainer, toast } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';

function App() {
  return (
    <div className="App">
      <ToastContainer position="top-right" autoClose={3000} />
      { /* ... reste du code */ }
    </div>
  );
}

// Utiliser dans vos composants
toast.success('Parcelle créée avec succès !');
toast.error('Erreur lors de la sauvegarde');
toast.info('Données chargées');
```



Rendre l'application responsive

Améliorer le CSS mobile

Dans `App.css`, ajouter des media queries :

css

```
/* Mobile first */
@media (max-width: 768px) {
  .navbar-menu {
    flex-direction: column;
    width: 100%;
  }

  .navbar-menu button {
    width: 100%;
    text-align: left;
  }

  .card-grid,
  .stats-grid {
    grid-template-columns: 1fr;
  }

  table {
    display: block;
    overflow-x: auto;
  }
}

/* Tablettes */
@media (min-width: 769px) and (max-width: 1024px) {
  .card-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}
```

Ajouter la synchronisation temps réel

Avec WebSockets

1. Backend - Socket.io :

```
bash
```

```
docker-compose exec backend npm install socket.io
```

javascript

```
const http = require('http');
const socketIo = require('socket.io');

const server = http.createServer(app);
const io = socketIo(server, {
  cors: { origin: '*' }
});

io.on('connection', (socket) => {
  console.log('Client connecté');

  socket.on('nouvelle-recolte', (data) => {
    io.emit('recolte-ajoutée', data);
  });
});

server.listen(PORT, () => {
  console.log(`Serveur sur port ${PORT}`);
});
```

2. Frontend :

bash

```
docker-compose exec frontend npm install socket.io-client
```

javascript

```
import io from 'socket.io-client';

const socket = io('http://localhost:3001');

socket.on('recolte-ajoutée', (data) => {
  console.log('Nouvelle récolte:', data);
  loadRecoltes(); // Recharger les données
});
```



Tableau de bord avancé

Créer des widgets personnalisés

Exemple de widget météo :

javascript

```
function WeatherWidget() {  
  const [weather, setWeather] = useState(null);  
  
  useEffect(() => {  
    fetch('https://api.openweathermap.org/data/2.5/weather?q=VotreVille&appid=VOTRE_CLE')  
      .then(res => res.json())  
      .then(data => setWeather(data));  
  }, []);  
  
  return (  
    <div className="card">  
      <h3>Météo du jour</h3>  
      {weather && (  
        <div>{weather.weather[0].description}</div>  
        <div>{Math.round(weather.main.temp - 273.15)}°C</div>  
      )}  
    </div>  
  );  
}
```

Ces exemples vous donnent une base solide pour personnaliser votre application. N'hésitez pas à adapter le code à vos besoins spécifiques !